Агрегатные функции

Теория

Что такое агрегатные функции.
Агрегатная функция берёт **много строк** и возвращает **одно значение**. Например, у нас есть таблица товаров — 100 строк. Мы хотим узнать, сколько всего товаров или среднюю цену. Именно для этого нужны агрегатные функции.

Основные агрегатные функции SQLite.

Функция	Что делает	Пример
`COUNT(*)`	Считает количество строк	`SELECT COUNT(*) FROM product;`
`COUNT(столбец)`	Считает количество **непустых** значений	`SELECT COUNT(category_id) FROM product;`
`SUM(столбец)`	Сумма значений	`SELECT SUM(price) FROM product;`
`AVG(столбец)`	Среднее арифметическое	`SELECT AVG(price) FROM product;`
`MIN(столбец)`	Минимальное значение	`SELECT MIN(price) FROM product;`
`MAX(столбец)`	Максимальное значение	`SELECT MAX(price) FROM product;`

Важно понимать разницу:

- `COUNT(*)` считает **все** строки, даже если в них есть NULL.
- `COUNT(столбец)` считает только строки, где в этом столбце **есть значение**.
- `SUM`, `AVG`, `MIN`, `MAX` **игнорируют NULL**.

Пример с таблицей.

id	name	price	category_id
1	Хлеб	40	1
2	Мыло	80	2
3	Молоко	60	1
4	Сыр	200	NULL
5	Чай	150	2

- `SELECT COUNT(*) FROM product;` → **5**.
- `SELECT COUNT(category_id) FROM product;` → **4**.
- `SELECT SUM(price) FROM product;` → **530**.
- `SELECT AVG(price) FROM product;` → **106**.
- `SELECT MIN(price) FROM product;` → **40**.
- `SELECT MAX(price) FROM product;` → **200**.

Группировка GROUP BY.
Агрегатные функции без группировки считают по всей таблице. Но часто нужно посчитать **по каждой категории отдельно**. Для этого используется `GROUP BY`.

```
SELECT category_id, COUNT(*), AVG(price)
FROM product
GROUP BY category_id;
```

Что произойдёт:

1. SQLite разобьёт строки на группы по `category_id`:
    - группа 1: Хлеб, Молоко;
    - группа 2: Мыло, Чай;
    - группа NULL: Сыр.
2. Внутри каждой группы посчитает `COUNT(*)` и `AVG(price)`.

Результат:

category_id	count	avg
1	2	50
2	2	115
NULL	1	200

Правило: в `SELECT` можно выводить только те столбцы, которые указаны в `GROUP BY`, и агрегатные функции.

Фильтрация групп HAVING.
`WHERE` фильтрует строки **до** группировки. `HAVING` фильтрует группы **после** группировки.

```
SELECT category_id, COUNT(*)
FROM product
GROUP BY category_id
HAVING COUNT(*) > 1;
```

Покажет только те категории, где больше одного товара.

Порядок выполнения запроса:

```
text
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
```

Типичные ошибки студентов:

- Путают `WHERE` и `HAVING`.
- Пытаются вывести столбец, которого нет в `GROUP BY`.
- Забывают, что `COUNT(столбец)` не считает NULL.

Разобранный пример

```sql
SELECT COUNT(*) FROM product;

SELECT AVG(price) FROM product;

SELECT category_id, COUNT(*)
FROM product
GROUP BY category_id;

SELECT category_id, SUM(price)
FROM product
GROUP BY category_id;

SELECT category_id, COUNT(*)
FROM product
GROUP BY category_id
HAVING COUNT(*) > 1;

SELECT MAX(price), MIN(price) FROM product;
```

Задачи

1. Сколько всего товаров?
2. Средняя цена.
3. Количество товаров по категориям.
4. Сумма цен по категориям.
5. Категории, где больше 2 товаров.
6. Самая дорогая и самая дешёвая цена.
7. Категории, где средняя цена > 100.
8. Товары, у которых цена выше средней по всей таблице.

Задача «со звёздочкой»:

9. Для каждой категории — количество товаров, средняя цена и максимальная цена. Отсортировать по средней цене.

Задача на поиск ошибки:

```sql
SELECT name, COUNT(*) FROM product GROUP BY category_id;
```

Почему этот запрос вызовет ошибку?

Задача на предсказание результата:

```sql
SELECT category_id, COUNT(*) FROM product GROUP BY category_id HAVING COUNT(*) > 1;
```

Данные: 3 товара в категории 1, 1 товар в категории 2.

Продвинутый случай:
Использование `COUNT(DISTINCT category_id)` — посчитать количество уникальных категорий.